

**UNIVERSIDADE DO ESTADO DO AMAZONAS
ESCOLA SUPERIOR DE TECNOLOGIA
GRUPO DE ESTUDOS APLICADOS À ROBÓTICA**

CÉLIO BENJAMIM



**CAPACITAÇÃO EM SISTEMAS EMBARCADOS:
ESP32**

**Manaus - AM
2025**



1. Introdução

ESP32 é uma pequena placa eletrônica que funciona como o cérebro de inúmeros projetos de automação e da Internet das Coisas (IoT). Poderoso, acessível e surpreendentemente fácil de usar, o ESP32 é a ferramenta perfeita que abre as portas para transformar suas ideias em realidade conectada.

2. O que é o ESP32?

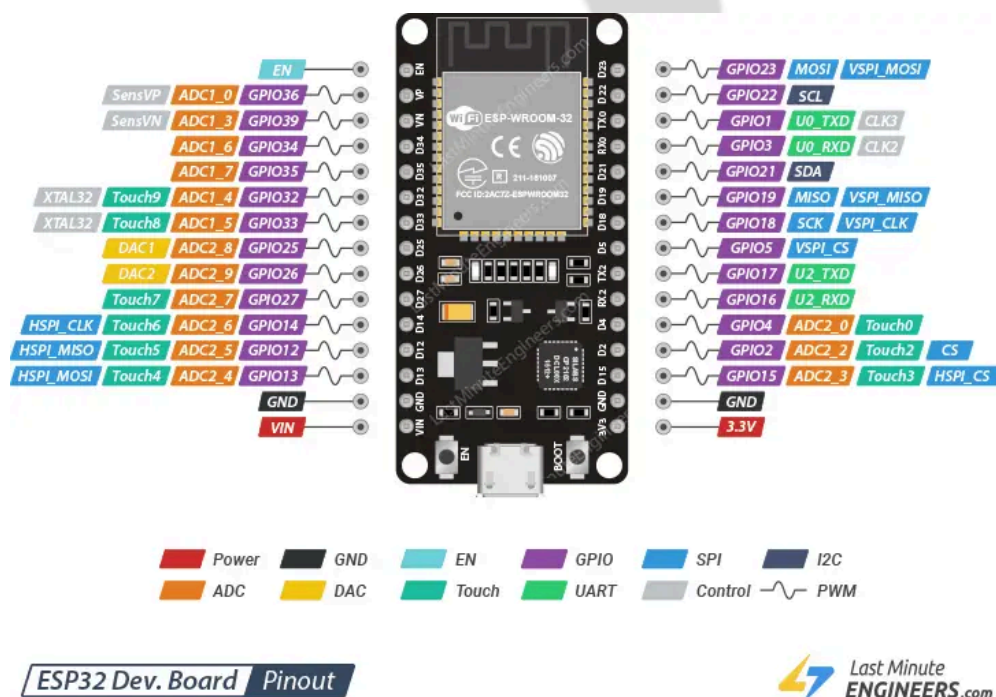


Figura 1: Pinout da ESP32

Em termos simples, o ESP32 (Figura 1) é um microcontrolador de baixo custo e baixo consumo de energia que funciona como um "sistema-em-um-chip" (SoC, System on a Chip), que utiliza um poderoso microprocessador Tensilica Xtensa LX6 e é fabricado pela TSMC usando seu processo de 40 nm. Criado em 2016 pela empresa chinesa Espressif Systems como o sucessor do popular ESP8266, ele integra não apenas o processador, mas também as duas tecnologias de comunicação sem fio mais importantes da atualidade: **Wi-Fi** e **Bluetooth**. Essa integração é sua característica mais marcante, pois permite que seus projetos se conectem facilmente a redes de internet e a outros dispositivos, como smartphones, sem a necessidade de componentes adicionais. Para entender o verdadeiro poder do ESP32, vamos comparar com o Arduino.



3. ESP32 vs. Arduino

Entender as diferenças entre o ESP32 e uma placa Arduino tradicional é a maneira mais rápida para compreender o salto de capacidade que o ESP32 representa. Essa comparação não apenas destaca seu poder superior, mas também revela uma de suas maiores vantagens: apesar de ser muito mais avançado, ele pode ser programado no mesmo ambiente familiar da Arduino IDE, tornando a transição muito suave.

A Tabela 1 resume as principais diferenças técnicas entre um ESP32 e um Arduino Uno:

Característica	Arduino Uno	ESP32
Processador	Single-Core 8-bit	Dual-Core (ou Single-Core) 32-bit
Velocidade (Clock)	16 MHz	Até 240 MHz
Memória SRAM	2 KB	520 KB
Conectividade	Nenhuma integrada (requer módulos)	Wi-Fi e Bluetooth integrados
Tensão de Operação	5V	3.3V

Tabela 1: Comparação ESP32 x Arduino UNO

Qual o impacto dessas diferenças?

- **Processador Dual-Core:** Ter dois núcleos é uma vantagem imensa para a IoT. O ESP32 pode dedicar um núcleo exclusivamente para gerenciar as tarefas de rede (Wi-Fi e Bluetooth), enquanto o outro núcleo fica totalmente livre para executar o seu código. Isso significa que seu programa principal



não será interrompido ou retardado pelas comunicações, resultando em projetos mais estáveis e responsivos.

- **Velocidade e Memória:** Com um clock até 15 vezes mais rápido e mais de 250 vezes a memória SRAM de um Arduino Uno, o ESP32 pode executar programas muito mais complexos. Ele pode processar dados de sensores com mais rapidez, executar servidores web mais robustos e lidar com tarefas que seriam impossíveis para microcontroladores mais simples.
- **Conectividade Integrada:** Esta é, talvez, a maior vantagem. Com um Arduino, adicionar Wi-Fi ou Bluetooth exige a compra de módulos extras, conhecidos como "shields", que aumentam o custo, o tamanho e a complexidade do projeto. O ESP32 já vem com tudo isso embutido, simplificando drasticamente o hardware e o software.

Apesar de todo esse poder, a comunidade de desenvolvedores garantiu que o ESP32 possa ser programado utilizando a conhecida e amigável **Arduino IDE**, o que o torna a atualização perfeita para quem já tem alguma experiência com Arduino.

4. Funcionalidades do ESP32

Além da potência de processamento, o verdadeiro valor do ESP32 reside em suas capacidades nativas de comunicação e interação com o mundo físico. Entender essas funcionalidades é o que permitirá a você explorar todo o potencial da placa em seus projetos de IoT. A seguir, detalhamos as funcionalidades que fazem do ESP32.

4.1. Conectividade Wi-Fi Versátil

O ESP32 não apenas se conecta ao Wi-Fi, mas o faz de duas maneiras distintas e muito úteis:

- **Modo Estação (Station Mode):** Neste modo, o ESP32 funciona como qualquer outro dispositivo em sua casa, como seu celular ou computador. Ele se conecta a um roteador Wi-Fi existente para acessar a internet. Este é o modo mais comum para projetos de IoT que precisam enviar dados para a nuvem ou ser controlados remotamente.
- **Modo Ponto de Acesso (Access Point Mode):** Neste modo, o ESP32 inverte os papéis e cria sua própria rede Wi-Fi. Outros dispositivos (como seu smartphone) podem se conectar diretamente a ele, sem a necessidade de um roteador intermediário. Isso é perfeito para configurar um dispositivo pela



primeira vez ou para projetos que precisam funcionar em locais sem uma rede Wi-Fi disponível.

4.2. Conectividade Bluetooth Dupla

O ESP32 inclui suporte para dois tipos de Bluetooth, cada um com uma finalidade específica:

- **Bluetooth Clássico:** É o tipo de Bluetooth que a maioria das pessoas conhece, usado para streaming de áudio para fones de ouvido ou para transferências de dados contínuas. A programação do Bluetooth Clássico no ESP32 é bem siml, funcionando como uma porta serial sem fio. É importante notar que os exemplos de Bluetooth Clássico, como o que veremos neste guia, são mais facilmente testados com dispositivos Android, já que o iOS possui requisitos mais rigorosos para perfis de Bluetooth Clássico.
- **Bluetooth Low Energy (BLE):** Como o nome sugere, esta é uma variante de baixíssimo consumo de energia. É ideal para dispositivos que funcionam a bateria e precisam enviar pequenas quantidades de dados periodicamente, como sensores de temperatura, pulseiras de fitness (wearables) e outros dispositivos de monitoramento.

4.3. Pinos de Entrada/Saída de Propósito Geral (GPIOs)

Os pinos GPIO (General Purpose Input/Output) são a ponte entre o ESP32 e o mundo físico. Eles permitem ler sensores e controlar atuadores como LEDs e motores. O ESP32 oferece uma vasta gama de periféricos que podem ser atribuídos a esses pinos, incluindo:

- **ADC (Conversor Analógico-Digital):** Permite ler sensores que fornecem valores variáveis em vez de um simples "ligado" ou "desligado". É usado para componentes como sensores de luminosidade, umidade do solo ou potenciômetros. No código, é o já conhecido `AnalogRead()`. O valor é lido numa resolução de 12 bits, ou seja, está no intervalo [0, 4095].
- **DAC (Conversor Digital-Analógico):** Faz o oposto do ADC, permitindo que o ESP32 gere sinais de tensão analógicos, úteis em projetos de áudio ou controle de motores. Os DACs internos do ESP32 operam com uma resolução de 8 bits, com valores de 0 a 255.
- **PWM (Modulação por Largura de Pulso):** Simula uma saída analógica através de pulsos digitais de diferentes durações. É ideal para controlar a velocidade de motores, o brilho de LEDs e servos. No ESP32, há duas abordagens principais para usar o PWM:



- **Função `analogWrite()`:** Para compatibilidade com a sintaxe do Arduino, pode-se usar a função `analogWrite()`, que permite definir um valor de ciclo de trabalho de 0 a 255 (resolução de 8 bits). O ESP32 possui múltiplos canais PWM, e a função `analogWrite()` usa automaticamente um canal disponível.
- **Controle LED (LEDC):** Esta é a abordagem nativa e mais poderosa do ESP32. O periférico LEDC oferece 16 canais PWM independentes, que podem ser configurados com frequências e resoluções de até 16 bits. Para usá-lo, você precisa primeiro configurar o canal com `ledcSetup()` e depois ajustar o ciclo de trabalho com `ledcWrite()`. A frequência de operação pode ser de até 40 MHz, dependendo da resolução configurada.
- **Sensores de Toque:** Alguns pinos do ESP32 são capacitivos e podem detectar o toque humano, permitindo criar interfaces de botão sem partes móveis.
- **Interfaces Padrão:** O ESP32 suporta todos os protocolos de comunicação comuns, como `I2C`, `SPI` e `UART`, garantindo compatibilidade com uma enorme variedade de sensores, telas e outros microcontroladores disponíveis no mercado.

Agora que você conhece a teoria por trás do poder do ESP32, é hora de colocar esse conhecimento em prática, começando pela configuração do seu ambiente de programação.

5. Configurando seu Ambiente

A teoria é importante, mas é na prática que a mágica realmente acontece. Esta seção é o guia prático e essencial para preparar seu computador para programar o ESP32. Utilizaremos a Arduino IDE.

Siga estes passos para configurar a Arduino IDE para programar o seu ESP32:

1. **Baixe e Instale a Arduino IDE:** Se você ainda não a tem, baixe a versão 2.x mais recente e estável do [site oficial do Arduino](https://www.arduino.cc/en/software) e instale-a em seu computador.
2. **Adicione a URL do Gerenciador de Placas:** Com a Arduino IDE aberta, vá em `Arquivo > Preferências`. No campo "URLs Adicionais de Gerenciadores de Placas", cole o seguinte endereço:



https://espressif.github.io/arduino-esp32/package_esp32_index.json

3. Clique em "OK" para salvar. Isso informa à IDE onde encontrar as informações sobre as placas ESP32.
4. **Instale o Pacote ESP32:** Agora, vá em **Ferramentas > Placa > Gerenciador de Placas**. Na janela que abrir, digite "esp32" na barra de pesquisa. Você verá um resultado da "Espressif Systems". Clique no botão "Instalar" para baixar e instalar o pacote de placas ESP32.
5. **Selecione a Placa Correta:** Após a instalação, feche o Gerenciador de Placas. Agora, no menu **Ferramentas > Placa**, você encontrará uma nova seção chamada "ESP32". Selecione uma placa genérica como a "**ESP32 Dev Module**", que é compatível com a maioria das placas de desenvolvimento ESP32.

Com seu ambiente configurado, você está pronto para carregar seu primeiro código e construir projetos incríveis. Vamos começar!

6. Projetos Práticos

Esta seção tem como objetivo transformar a teoria em resultados. Os projetos a seguir foram cuidadosamente escolhidos para construir seu conhecimento de forma progressiva. Começaremos com um conceito familiar do Arduino — controlar um LED — e o vamos elevar ao nível da IoT, para depois explorarmos a simplicidade da comunicação Bluetooth do ESP32.

6.1. O "Hello, World!" do Hardware via Wi-Fi

Este projeto é o equivalente ao "Blink" (piscar um LED) do mundo da IoT. Em vez de apenas fazer um LED piscar em um intervalo fixo, você irá controlá-lo remotamente através de uma página da web servida pelo próprio ESP32.

Vamos ver o exemplo da própria biblioteca **WiFi.h**. No Arduino IDE, acesse o menu **Arquivo > Exemplos > WiFi > SimpleWifiServer**. Ao abrir o exemplo você deve mudar algumas coisas: (1) Mude o SSID e a senha para a rede que quer se conectar (exemplo, `ssid = "GEAR_IOT"; password = "gearingear";`); (2) Mude o pino do LED que está conectado, por exemplo, queremos controlar o LED interno da ESP (LED_BUILTIN), ele está no pino 2, então devemos mudar o pino que está sendo controlado para o pino 2, pode ser feito um `#define LED 2`.



Análise do Código:

1. **Bibliotecas e Configurações:** Incluímos a biblioteca `WiFi.h`. As variáveis `ssid` e `password` armazenam as credenciais da sua rede. A definição `#define LED_BUILTIN 2` é particularmente útil, pois o pino 2 corresponde ao LED azul integrado na maioria das placas de desenvolvimento ESP32, permitindo testar o código sem nenhum componente externo. `WiFiServer server(80);` cria um objeto de servidor que "escuta" por conexões na porta 80.
2. **setup():** O código inicializa a comunicação serial (para depuração), configura o pino do LED e inicia a conexão Wi-Fi. Após conectar, ele imprime o endereço IP do ESP32 no Monitor Serial e inicia o servidor com `server.begin()`.
3. **loop():** O coração do programa. Ele espera por um cliente (`client.available()`). Quando um navegador acessa o IP do ESP32, o código envia uma página HTML simples com dois links.
4. **O Controle Físico:** A parte mais importante é `if (currentLine.endsWith("GET /H"))`. Quando você clica no link para ligar o LED, seu navegador faz uma requisição para o endereço `IP_DO_ESP32/H`. O código detecta essa requisição e executa o comando `digitalWrite(LED, HIGH);`, ligando o LED. O mesmo acontece para desligar com `/L`.

Após carregar o código, abra o Monitor Serial (**Ferramentas > Monitor Serial**) com a velocidade de **115200** (certifique-se de que a velocidade no canto inferior direito do Monitor Serial esteja configurada para **115200** para ver a saída corretamente). Anote o endereço de IP que aparecer, digite-o em um navegador na mesma rede Wi-Fi e controle seu primeiro dispositivo IoT!

6.2. Sua Primeira Comunicação Sem Fio com Bluetooth

Este projeto demonstra a simplicidade de usar o Bluetooth Clássico no ESP32 para uma tarefa fundamental: trocar mensagens de texto entre o ESP32 e um smartphone, de forma semelhante a uma conversa por chat.



Novamente vamos usar um exemplo da biblioteca da ESP32, vá em [Arquivo > Exemplos > BluetoothSerial > SerialToSerialBT](#), que é um exemplo padrão da biblioteca [BluetoothSerial](#).

Como Testar o Projeto:

1. Carregue o código para o seu ESP32.
2. Abra o Monitor Serial na Arduino IDE com a velocidade (baud rate) de **115200**. Você verá a mensagem "O dispositivo foi iniciado...".
3. No seu smartphone Android, vá até a Play Store e instale um aplicativo de terminal Bluetooth. O "Serial Bluetooth Terminal" é uma excelente opção gratuita.
4. No aplicativo, procure por novos dispositivos Bluetooth e pareie com o dispositivo chamado **"ESP32-BT-Slave"**.
5. Após o pareamento, conecte-se ao "ESP32-BT-Slave" dentro do aplicativo.
6. Agora, a mágica acontece:
 - Digite uma mensagem no aplicativo e pressione "enviar". O texto aparecerá instantaneamente no seu Monitor Serial na Arduino IDE.
 - Digite uma mensagem na barra de entrada do Monitor Serial e pressione "Enter" ou clique em "Enviar". O texto aparecerá no aplicativo em seu celular.

Você acabou de criar uma ponte de comunicação serial sem fio, uma base fundamental para inúmeros projetos interativos.

7. Conclusão

Ao longo deste guia, você descobriu o poder e a acessibilidade do ESP32. Recapitulamos suas principais vantagens: um processador dual-core robusto, conectividade Wi-Fi e Bluetooth integrada em um único chip e, o mais importante, uma programação simplificada através da familiar Arduino IDE. Você deu os primeiros passos práticos, controlando um LED pela internet e estabelecendo uma comunicação sem fio com seu smartphone, provando que a Internet das Coisas está ao seu alcance.

Esta é apenas a ponta do iceberg. À medida que você se sentir mais confortável, poderá explorar conceitos mais avançados, como o protocolo **ESP-NOW**, um protocolo de comunicação direta e de alta velocidade entre placas ESP32 que funciona sem a necessidade de um roteador Wi-Fi, ou aprender a utilizar os **dois núcleos** do processador para executar tarefas complexas e comunicação de rede simultaneamente sem comprometer o desempenho.



Encare o ESP32 não como um componente eletrônico complexo, mas como um parceiro criativo pronto para dar vida às suas ideias. Experimente os projetos que apresentamos, modifique-os, combine-os e comece a construir as soluções conectadas que você sempre imaginou. Sua jornada no fascinante mundo da Internet das Coisas está apenas começando.